

CleanNotes（清记）后端开发文档 v1.3.0

本文档基于 CleanNotes v1.3.0 现有代码反向工程整理，涵盖数据库设计、RLS 安全策略、认证系统、数据同步机制、迁移历史及 API 接口清单。

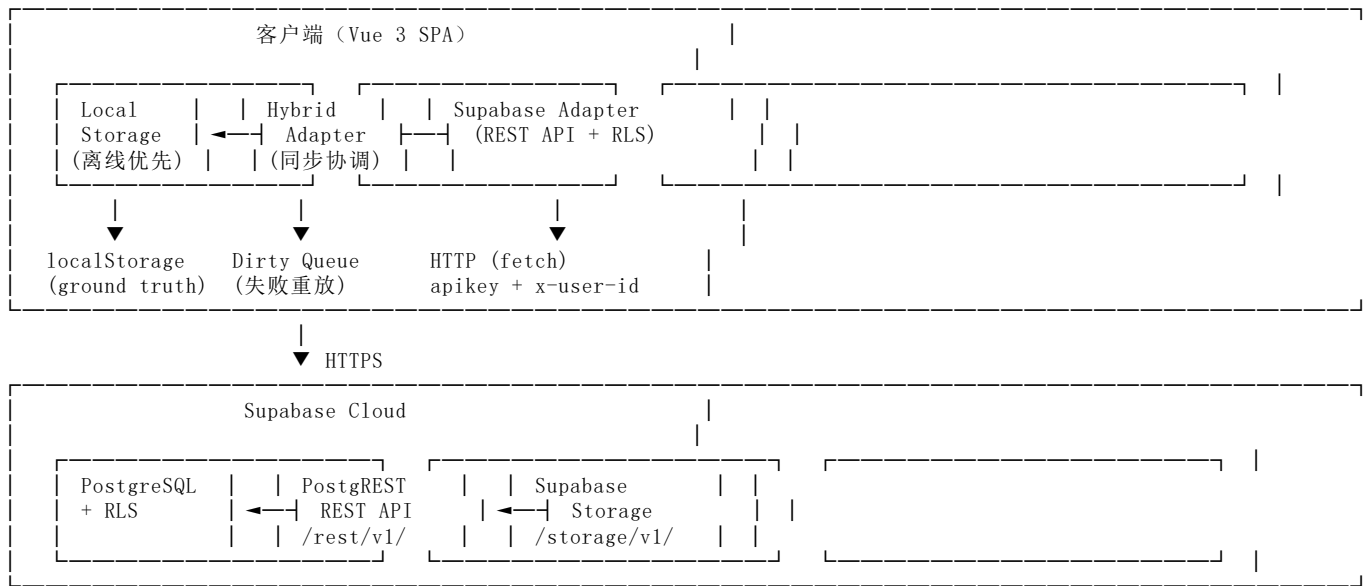
1. 技术架构概述

1.1 Supabase BaaS 选型理由

CleanNotes 后端采用 **Supabase**（开源 Firebase 替代方案）作为 BaaS（Backend as a Service）平台，核心选型理由：

维度	选型	理由
数据库	PostgreSQL	关系型数据库，支持 JSONB、CHECK 约束、行级安全（RLS）
API 层	PostgREST（Supabase 内置）	自动将 PostgreSQL 表暴露为 RESTful API，无需手写后端代码
安全	Row Level Security（RLS）	数据库层面行级隔离，按 <code>user_id</code> 过滤，无需应用层鉴权
存储	Supabase Storage	文件上传/公开读取，支持 Bucket 级 RLS 策略
认证	自建认证（手机号 + PBKDF2 密码）	MVP 阶段不依赖 Supabase Auth，使用 anon key + 自定义用户表
实时	暂未使用	v1.3.0 采用轮询同步，未启用 Supabase Realtime

1.2 架构组合

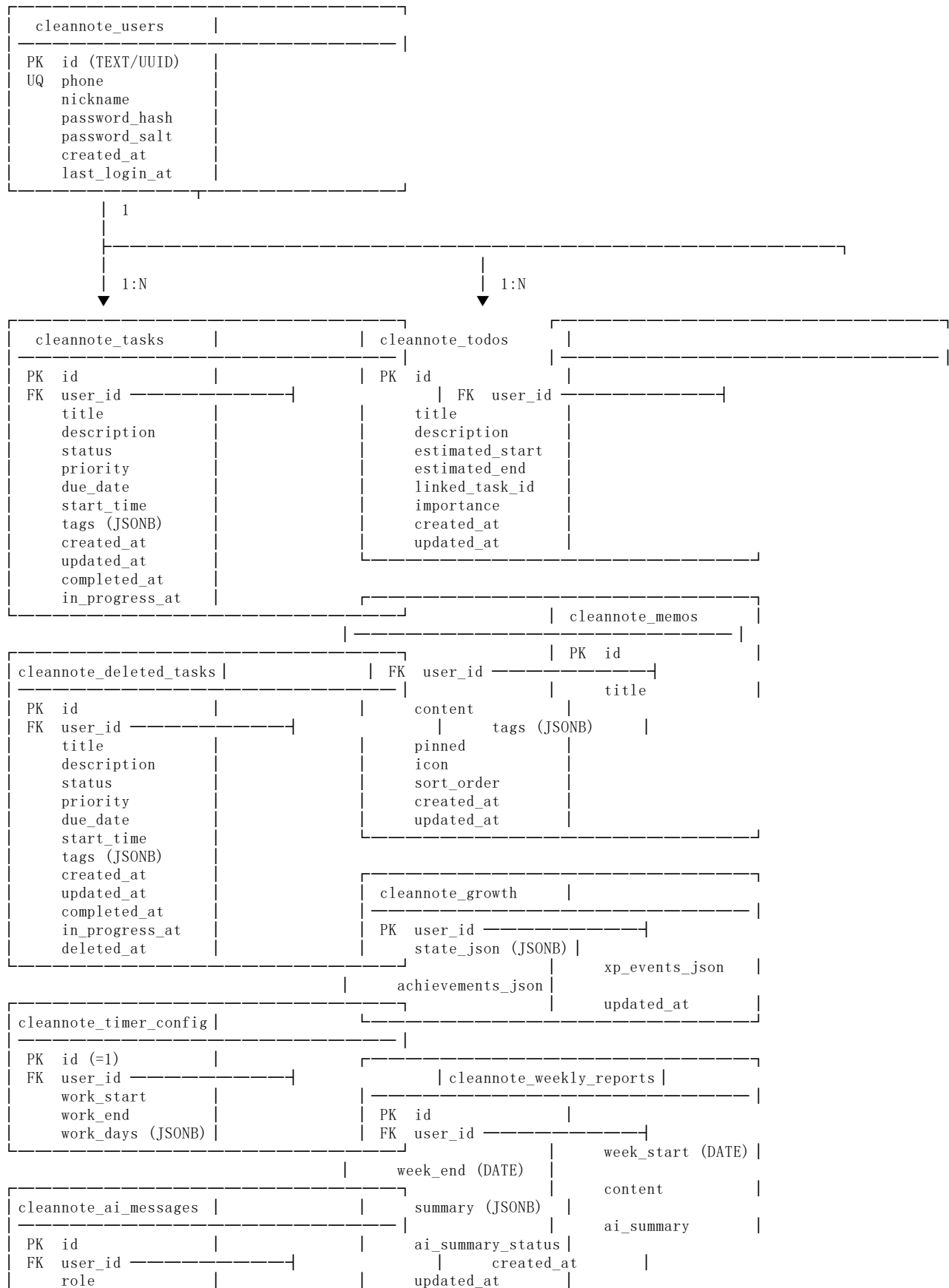


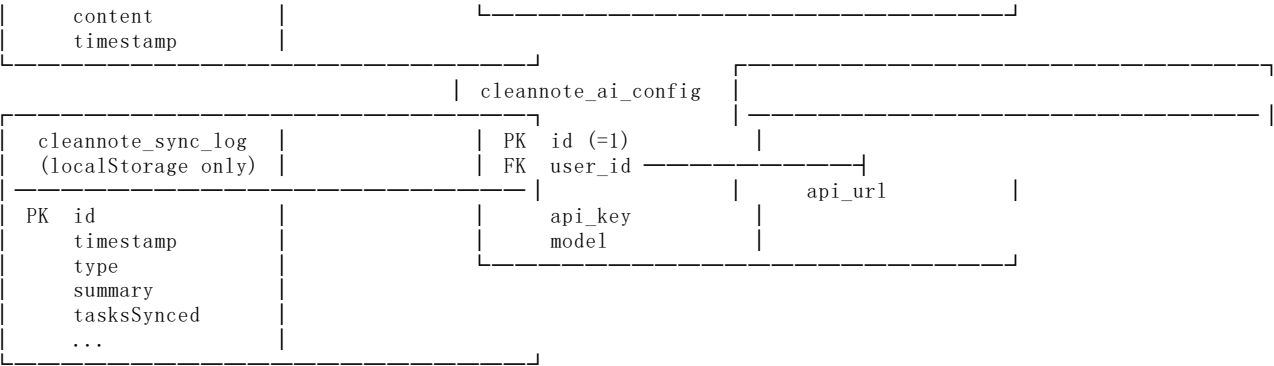
1.3 关键设计决策

- **离线优先：**所有写操作先写 `localStorage`（本地为 `ground truth`），再异步同步到 Supabase
- **增量同步：**单记录粒度 `upsert/delete`，不再全量替换
- **anon key + x-user-id header：**不使用 Supabase Auth JWT，通过自定义 header 传递用户身份给 RLS 策略
- **密码客户端哈希：**PBKDF2 派生在浏览器完成，服务端仅存储哈希+盐，无法还原明文

2. 数据库设计

2.1 ER 图 (ASCII 实体关系图)





关系说明： - cleannote_users 是所有表的外键根节点，ON DELETE CASCADE 级联删除 - cleannote_tasks 与 cleannote_deleted_tasks 为逻辑分离（软删除→回收站），非外键关联 - cleannote_growth 以 user_id 为主键（每用户一行），存储 JSONB 聚合数据 - cleannote_timer_config 和 cleannote_ai_config 使用固定 id=1 + UNIQUE(user_id) 约束（每用户一行） - cleannote_sync_log 仅存在于 localStorage，不落库 Supabase

2.2 核心表结构

2.2.1 cleannote_users（用户表）

字段名	类型	约束	默认值	说明
id	TEXT	PRIMARY KEY	—	UUID，crypto.randomUUID() 生成
phone	TEXT	UNIQUE NOT NULL	—	手机号，唯一标识
nickname	TEXT	NOT NULL	''	昵称，默认 用户{手机后4位}
password_hash	TEXT	—	NULL	PBKDF2-HMAC-SHA256 派生哈希（base64），NULL=未设密
password_salt	TEXT	—	NULL	每用户随机盐（base64，16字节），NULL=未设密
created_at	TIMESTAMPTZ	NOT NULL	now()	注册时间
last_login_at	TIMESTAMPTZ	NOT NULL	now()	最后登录时间，每次登录 PATCH 更新

来源： supabase-schema.sql:11-21， migration-password.sql 补充 password_hash/password_salt 列

2.2.2 cleannote_tasks（任务表）

字段名	类型	约束	默认值	说明
id	TEXT	PRIMARY KEY	—	任务 UUID
user_id	TEXT	NOT NULL, FK → users(id) CASCADE	—	所属用户
title	TEXT	NOT NULL	—	任务标题
description	TEXT	NOT NULL	''	任务描述
status	TEXT	NOT NULL, CHECK IN ('todo','in_progress','done')	'todo'	任务状态
priority	TEXT	NOT NULL, CHECK IN ('low','medium','high')	'medium'	优先级
due_date	TEXT	—	NULL	截止日期（YYYY-MM-DD）
start_time	TEXT	—	NULL	计划开始时间（HH:mm）
tags	JSONB	NOT NULL	'[]'	标签数组

字段名	类型	约束	默认值	说明
created_at	TIMESTAMPTZ	NOT NULL	now()	创建时间
updated_at	TIMESTAMPTZ	NOT NULL	now()	更新时间
completed_at	TIMESTAMPTZ	—	NULL	完成时间
in_progress_at	TIMESTAMPTZ	—	NULL	进入"进行中"时间（v5新增）

索引：idx_tasks_user_id ON cleannote_tasks(user_id)

来源：supabase-schema.sql:24-38。 in_progress_at 为 v4→v5 迁移新增字段。

2.2.3 cleannote_todos（待办表）

字段名	类型	约束	默认值	说明
id	TEXT	PRIMARY KEY	—	待办 UUID
user_id	TEXT	NOT NULL, FK → users(id)	—	所属用户
title	TEXT	NOT NULL	—	待办标题
description	TEXT	—	''	描述
estimated_start	TEXT	—	NULL	预计开始日期（YYYY-MM-DD）
estimated_end	TEXT	—	NULL	预计结束日期（YYYY-MM-DD）
linked_task_id	TEXT	—	NULL	转任务后关联的任务 ID
importance	INTEGER	—	0	重要等级 1-5，0=未评级
created_at	TIMESTAMPTZ	—	now()	创建时间
updated_at	TIMESTAMPTZ	—	now()	更新时间

索引：idx_todos_user_id、idx_todos_updated_at

来源：migration-todos-growth.sql:7-17

2.2.4 cleannote_memos（备忘录表）

字段名	类型	约束	默认值	说明
id	TEXT	PRIMARY KEY	—	备忘 UUID
user_id	TEXT	NOT NULL	—	所属用户
title	TEXT	NOT NULL	—	标题
content	TEXT	NOT NULL	''	内容（HTML 富文本）
tags	JSONB	NOT NULL	'[]'	标签数组
pinned	BOOLEAN	NOT NULL	false	是否置顶
icon	TEXT	NOT NULL	''	页面图标（emoji）
sort_order	INTEGER	NOT NULL	0	排序权重，越小越靠前
created_at	TIMESTAMPTZ	NOT NULL	—	创建时间
updated_at	TIMESTAMPTZ	NOT NULL	—	更新时间

索引：idx_memos_user_id、idx_memos_updated_at、idx_memos_sort_order

来源：migration-memos.sql:4-15。 icon 和 sort_order 为后续 ALTER TABLE 补充列。

2.2.5 cleannote_deleted_tasks（回收站表）

字段名	类型	约束	默认值	说明
id	TEXT	PRIMARY KEY	—	任务 UUID（与原任务同 ID）
user_id	TEXT	NOT NULL, FK → users(id) CASCADE	—	所属用户
title	TEXT	NOT NULL	—	任务标题
description	TEXT	NOT NULL	''	任务描述
status	TEXT	NOT NULL, CHECK IN ('todo','in_progress','done')	'todo'	任务状态
priority	TEXT	NOT NULL, CHECK IN ('low','medium','high')	'medium'	优先级
due_date	TEXT	—	NULL	截止日期
start_time	TEXT	—	NULL	计划开始时间
tags	JSONB	NOT NULL	'[]'	标签数组
created_at	TIMESTAMPTZ	NOT NULL	now()	原创建时间
updated_at	TIMESTAMPTZ	NOT NULL	now()	更新时间
completed_at	TIMESTAMPTZ	—	NULL	完成时间
in_progress_at	TIMESTAMPTZ	—	NULL	进行中时间（v5新增）
deleted_at	TIMESTAMPTZ	NOT NULL	now()	删除时间

索引：idx_deleted_tasks_user_id

来源：supabase-schema.sql:41-56。结构与 cleannote_tasks 完全一致，额外增加 deleted_at 字段。

2.2.6 cleannote_growth（养成数据表）

字段名	类型	约束	默认值	说明
user_id	TEXT	PRIMARY KEY, FK → users(id)	—	用户 ID（每用户一行）
state_json	JSONB	NOT NULL	' {} '	GrowthState 完整状态
xp_events_json	JSONB	NOT NULL	' [] '	XpEvent[] 经验事件列表
achievements_json	JSONB	NOT NULL	' [] '	AchievementRecord[] 成就记录
updated_at	TIMESTAMPTZ	—	now()	更新时间

GrowthState JSONB 结构：

```
{
  "level": 1,
  "xp": 0,
  "totalXp": 0,
  "streakDays": 0,
  "maxStreakDays": 0,
  "dailyState": "vitality | withered | recovery",
  "witheredDays": 0,
  "lastActiveDate": "",
  "lastXpGainAt": ""
}
```

来源：migration-todos-growth.sql:49-55。采用 JSONB 聚合存储，避免多表关联查询。

2.2.7 cleannote_achievements（成就表）

注意：代码中 AchievementRecord 类型存在 ({ id, unlockedAt })，但没有独立的 cleannote_achievements 表。成就记录以 JSONB 数组形式内嵌在 cleannote_growth.achievements_json 字段中。growthStorage.ts 负责本地管理，同步时整体 upsert 到 cleannote_growth 表。

2.2.8 cleannote_weekly_reports (周报表)

字段名	类型	约束	默认值	说明
id	TEXT	PRIMARY KEY	—	周报 UUID
user_id	TEXT	NOT NULL	—	所属用户
week_start	DATE	NOT NULL	—	周一日期 (YYYY-MM-DD)
week_end	DATE	NOT NULL	—	周日日期 (YYYY-MM-DD)
content	TEXT	—	''	周报内容 (HTML 富文本)
summary	JSONB	—	' {} '	统计摘要
ai_summary	TEXT	—	NULL	AI 智能总结
ai_summary_status	TEXT	—	NULL, CHECK IN ('generating','success','failed')	AI 总结状态
created_at	TIMESTAMPTZ	—	now ()	创建时间
updated_at	TIMESTAMPTZ	—	now ()	更新时间

索引：idx_weekly_reports_user_id、idx_weekly_reports_week_start

WeeklyReportSummary JSONB 结构：

```
{
  "tasksCreated": 0,
  "tasksCompleted": 0,
  "todosCreated": 0,
  "totalXpGained": 0,
  "completionRate": 0,
  "streakDays": 0
}
```

来源：migration-weekly-reports.sql:7-16, migration-ai-summary.sql 补充 ai_summary/ai_summary_status 列

2.2.9 cleannote_sync_log (同步日志表)

注意：同步日志**不落库 Supabase**，仅存储在 localStorage，key 格式为 cleannotes_sync_logs_{userId}。最多保留 3 条，FIFO 淘汰。

字段名	类型	说明
id	string	日志 ID ({timestamp}_{random})
timestamp	string	UTC ISO 时间戳
type	string	'full_merge' 'incremental' 'dirty_replay' 'error'
summary	string	摘要描述
tasksSynced	number	同步的任务数
deletedSynced	number	同步的回收站数
memosSynced	number	同步的备忘录数

字段名	类型	说明
todosSynced	number	同步的待办数
reportsSynced	number	同步的周报数
status	string	'success' 'partial' 'failed'
errorMsg	string?	错误信息

来源：src/services/syncLog.ts

2.2.10 其他表

cleannote_timer_config (上下班倒计时配置):

字段名	类型	约束	默认值	说明
id	INT	PRIMARY KEY, CHECK (=1)	1	固定为 1 (单例)
user_id	TEXT	NOT NULL, UNIQUE, FK → users(id)	—	所属用户
work_start	TEXT	NOT NULL	'09:00'	上班时间 (HH:mm)
work_end	TEXT	NOT NULL	'18:00'	下班时间 (HH:mm)
work_days	JSONB	NOT NULL	'[1, 2, 3, 4, 5]'	工作日数组 (1=周一...7=周日)

来源：supabase-schema.sql:59-65

cleannote_ai_messages (AI 对话记录):

字段名	类型	约束	默认值	说明
id	TEXT	PRIMARY KEY	—	消息 UUID
user_id	TEXT	NOT NULL, FK → users(id) CASCADE	—	所属用户
role	TEXT	NOT NULL, CHECK IN ('user','assistant','system')	—	角色
content	TEXT	NOT NULL	—	消息内容
timestamp	TIMESTAMPTZ	NOT NULL	now()	时间戳

来源：supabase-schema.sql:68-74

cleannote_ai_config (AI API 配置):

字段名	类型	约束	默认值	说明
id	INT	PRIMARY KEY, CHECK (=1)	1	固定为 1 (单例)
user_id	TEXT	NOT NULL, UNIQUE, FK → users(id)	—	所属用户
api_url	TEXT	NOT NULL	''	AI API 地址
api_key	TEXT	NOT NULL	''	API 密钥
model	TEXT	NOT NULL	''	模型名称

来源：supabase-schema.sql:77-83

2.3 时间戳规范

2.3.1 存储规范

所有写入 Supabase TIMESTAMPTZ 字段的时间戳必须使用 UTC ISO 格式 (带 Z 后缀):

PostgreSQL 的 `TIMESTAMPZ` 类型会将带 `Z` 后缀的时间戳正确解析为 UTC 存储。

2.3.2 核心函数

来源: `src/utils/time.ts`

函数	签名	用途
<code>toUTCISO(date?)</code>	<code>→ string</code>	生成 UTC ISO 时间戳 (<code>new Date().toISOString()</code>)，用于 Supabase 存储
<code>toLocalISO(date?)</code>	<code>→ string</code>	生成本地时间 ISO 字符串 (无 Z 后缀)，仅用于 <code>datetime-local</code> 输入框
<code>toLocalDate(date?)</code>	<code>→ string</code>	生成本地日期字符串 (YYYY-MM-DD)
<code>normalizeTimestamp(ts)</code>	<code>→ string null</code>	防护函数 ：将历史遗留时间戳修正为 UTC ISO

2.3.3 `normalizeTimestamp()` 防护逻辑

输入 `ts`
└—— 已有 `Z` 后缀 或 时区偏移 (+08:00) → 原样返回 (已是合法 UTC)
└—— 无时区后缀 (legacy `toLocalISO` 格式) → 解析为本地时间，转 UTC ISO
└—— 无法解析 → 原样返回 (容错)

此函数在所有 `taskToRow()`、`deletedTaskToRow()`、`aiMsgToRow()` 等行映射函数中被调用，确保写入 Supabase 前时间戳格式正确。

2.3.4 本地显示转换规则

前端显示时使用 `new Date(ts).getHours()` 等 API，JavaScript `Date` 会自动将 UTC ISO 转换为本地时区。`datetime-local` 输入框使用 `toLocalISO()` 生成无时区后缀的本地时间字符串。

2.3.5 历史时区问题与修复

旧版代码使用 `toLocalISO()` 生成无 `Z` 后缀的本地时间 (如 CST 2026-07-01T12:34:00.000)，PostgreSQL 将其当作 UTC 存储，导致数据库时间比真实 UTC 多 8 小时。修复方案见 [7. 数据迁移历史](#) 中的 `fix-supabase-timezone.sql`。

2.4 camelCase / snake_case 映射

前端 TypeScript 使用 camelCase，数据库使用 snake_case。映射在 `src/services/supabase.ts` 中的 `*ToRow()` / `rowTo*()` 函数完成。

2.4.1 Task 映射

前端 (camelCase)	数据库 (snake_case)	备注
<code>id</code>	<code>id</code>	不变
<code>title</code>	<code>title</code>	不变
<code>description</code>	<code>description</code>	不变
<code>status</code>	<code>status</code>	不变
<code>priority</code>	<code>priority</code>	不变
<code>dueDate</code>	<code>due_date</code>	
<code>startDate</code>	<code>start_date</code>	代码中映射但 schema 未定义此列
<code>startTime</code>	<code>start_time</code>	

前端 (camelCase)	数据库 (snake_case)	备注
tags	tags	JSONB, 写入时直接传数组
createdAt	created_at	经 normalizeTimestamp() 处理
updatedAt	updated_at	经 normalizeTimestamp() 处理
completedAt	completed_at	经 normalizeTimestamp() 处理
inProgressAt	in_progress_at	经 normalizeTimestamp() 处理
—	user_id	由 currentUserId 注入, 前端对象无此字段

2.4.2 Todo 映射

前端	数据库	备注
estimatedStart	estimated_start	
estimatedEnd	estimated_end	
linkedTaskId	linked_task_id	
importance	importance	不变
createdAt	created_at	
updatedAt	updated_at	

2.4.3 Memo 映射

前端	数据库	备注
pinned	pinned	不变
icon	icon	不变
sortOrder	sort_order	
createdAt	created_at	
updatedAt	updated_at	

2.4.4 WeeklyReport 映射

前端	数据库	备注
weekStart	week_start	
weekEnd	week_end	
content	content	不变
summary	summary	JSONB
aiSummary	ai_summary	
aiSummaryStatus	ai_summary_status	

2.4.5 Growth 映射

Growth 表不使用逐字段映射, 而是整体 JSONB 存储:

前端对象	数据库字段	备注
GrowthState	state_json	完整 JSONB 对象
XpEvent[]	xp_events_json	JSONB 数组

前端对象	数据库字段	备注
AchievementRecord[]	achievements_json	JSONB 数组

2.4.6 TimerConfig / AiConfig / AiMessage 映射

前端	数据库
workStart	work_start
workEnd	work_end
workDays	work_days (JSON.stringify)
apiUrl	api_url
apiKey	api_key
model	model
timestamp (AiMessage)	timestamp

注意：tags 和 workDays 在写入时为 JSONB，PostgREST 自动处理。读取时需判断：若返回 string 则 JSON.parse，若已为对象则直接使用。

3. RLS 安全策略

3.1 行级安全概述

所有业务表均启用 PostgreSQL Row Level Security (RLS)。RLS 在数据库层面强制执行行级过滤，确保用户只能访问自己的数据。

3.2 x-user-id header 机制

```
客户端请求
|-- apikey: {SUPABASE_KEY}           ← anon key
|-- Authorization: Bearer {SUPABASE_KEY}
|-- Content-Type: application/json
|-- x-user-id: {currentUserId}       ← 自定义 header, RLS 据此过滤
```

RLS 策略通过 PostgreSQL 的 current_setting('request.headers', true)::json ->> 'x-user-id' 提取 header 值，与行中的 user_id (或 id) 比较。

来源：src/services/supabase.ts:8-16 (buildHeaders())

3.3 各表 RLS 策略

3.3.1 早期表 (FOR ALL 策略)

supabase-schema.sql 中定义的表使用 FOR ALL 单策略，同时覆盖 SELECT/INSERT/UPDATE/DELETE：

表名	策略名	匹配条件
cleannote_users	Users can access own profile	id = x-user-id
cleannote_tasks	Users can access own tasks	user_id = x-user-id
cleannote_deleted_tasks	Users can access own deleted tasks	user_id = x-user-id
cleannote_timer_config	Users can access own timer config	user_id = x-user-id
cleannote_ai_messages	Users can access own AI messages	user_id = x-user-id

表名	策略名	匹配条件
cleannote_ai_config	Users can access own AI config	user_id = x-user-id
cleannote_memos	Users can access own memos	user_id = x-user-id

策略语法（以 tasks 为例）：

```
CREATE POLICY "Users can access own tasks" ON cleannote_tasks
FOR ALL
USING (user_id = current_setting('request.headers', true)::json ->> 'x-user-id')
WITH CHECK (user_id = current_setting('request.headers', true)::json ->> 'x-user-id');
```

- USING：控制 SELECT / UPDATE / DELETE 可见的行
- WITH CHECK：控制 INSERT / UPDATE 允许写入的行

3.3.2 后期表（分操作策略）

migration-todos-growth.sql 和 migration-weekly-reports.sql 中定义的表使用分操作策略，每种操作独立：

表名	SELECT	INSERT	UPDATE	DELETE
cleannote_todos	todos_select	todos_insert	todos_update	todos_delete
cleannote_growth	growth_select	growth_insert	growth_update	growth_delete
cleannote_weekly_reports	weekly_reports_select	weekly_reports_insert	weekly_reports_update	weekly_reports_del

所有策略均以 user_id = current_setting('request.headers', true)::json ->> 'x-user-id' 作为过滤条件。

3.4 安全说明

当前方案使用 anon key + x-user-id header，安全性取决于客户端正确传递 user_id。后续可升级为 Supabase Auth + JWT 实现更强的服务端鉴权。此方案在 MVP 阶段足够，因为：

1. 用户无法伪造他人 user_id（需要知道对方 UUID）
2. RLS 在数据库层面强制过滤，即使 API 层被绕过
3. x-user-id 由 currentUserId（内存变量）注入，仅在登录后设置

4. Supabase Storage

4.1 cleannote_attachments Bucket

属性	值
Bucket 名称	cleannote_attachments
公开读取	是（Public bucket）
文件大小限制	10MB（建议）
允许的 MIME 类型	image/*、application/pdf、text/plain、Office 文档、application/zip、application/x-rar-compressed

4.2 文件路径规则

```
memo/{userId}/{timestamp}-{safeFileName}
```

示例：memo/abc-123-uuid/1719800000000-image.png

来源：src/services/supabase.ts:544-548 (supabaseUploadAttachment())

4.3 Storage RLS 策略

来源：migration-attachments-storage.sql

策略名	操作	条件
Allow public read on cleannote_attachments	SELECT	bucket_id = 'cleannote_attachments' (公开读取，无需 x-user-id)
Allow user upload to own memo folder	INSERT	bucket_id = 'cleannote_attachments' AND foldername(name)[1] = 'memo' AND foldername(name)[2] = x-user-id
Allow user delete own attachments	DELETE	bucket_id = 'cleannote_attachments' AND foldername(name)[2] = x-user-id

4.4 当前状态

重要：当前 v1.3.0 中附件以 **base64 内嵌**在备忘录 content 字段中 (HTML)，Supabase Storage 代码已实现但**尚未在实际流程中使用**。supabaseUploadAttachment()、supabaseGetPublicUrl()、supabaseDeleteAttachment() 函数已就绪，可在后续版本启用。

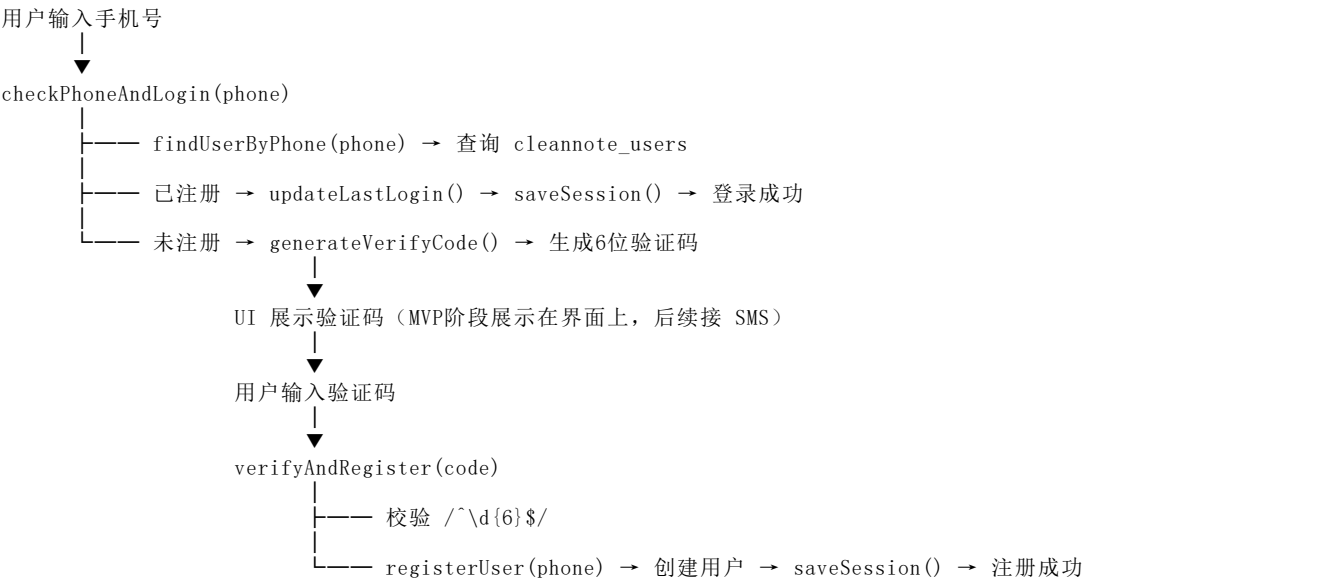
4.5 Storage API 端点

操作	方法	URL
上传文件	POST	{SUPABASE_URL}/storage/v1/object/cleannote_attachments/{path}
获取公开 URL	GET	{SUPABASE_URL}/storage/v1/object/public/cleannote_attachments/{path}
删除文件	DELETE	{SUPABASE_URL}/storage/v1/object/cleannote_attachments/{path}

5. 认证系统

5.1 手机号登录流程（验证码模式）

来源：src/stores/auth.ts、src/services/auth.ts



验证码生成 (src/services/auth.ts:192-194):

```
export function generateVerifyCode(): string {
  return String(Math.floor(100000 + Math.random() * 900000))
}
```

MVP 阶段验证码展示在界面上，任何6位数字均可通过校验。

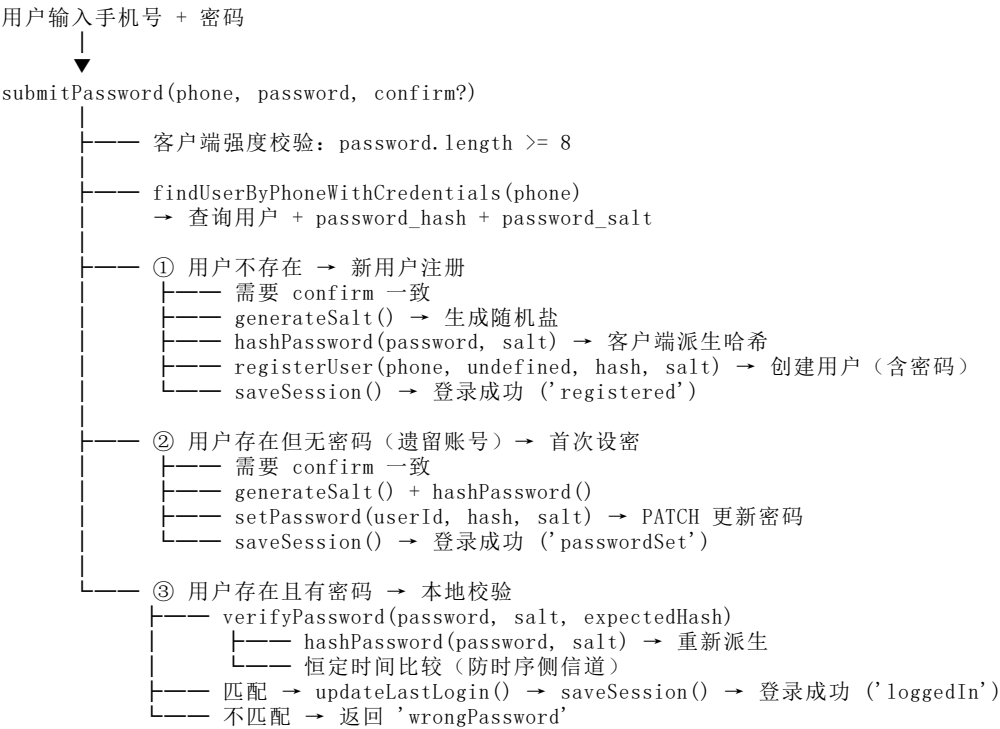
5.2 密码认证 (PBKDF2-HMAC-SHA256)

来源: src/utils/crypto.ts、src/stores/auth.ts:submitPassword()

5.2.1 密码哈希参数

参数	值	说明
算法	PBKDF2-HMAC-SHA256	OWASP 推荐算法
迭代次数	210,000	OWASP 2023 推荐值
盐长度	16 字节	每用户随机盐
派生位数	256 bits (32 字节)	输出哈希长度
编码	base64	哈希和盐均以 base64 存储

5.2.2 密码登录流程



5.2.3 安全要点

- 密码明文永不离开客户端: 仅 PBKDF2 派生哈希+盐上传至服务端
- 恒定时间比较: verifyPassword() 使用逐字节 XOR 比较, 防止时序侧信道
- 需要安全上下文: crypto.subtle 要求 HTTPS / localhost / Tauri webview
- 遗留账号兼容: password_hash 为 NULL 的旧账号在首次密码登录时被引导设置密码

5.3 会话管理

来源：src/services/auth.ts:21-56

5.3.1 Session 结构

```
interface Session {
  userId: string
  phone: string
  nickname: string
  loginAt: number // Unix timestamp (ms)
  expiresAt: number // Unix timestamp (ms)
}
```

5.3.2 存储与有效期

属性	值
存储 key	cleannote_session (localStorage)
有效期	7 天 (7 * 24 * 60 * 60 * 1000 ms)
恢复逻辑	getSession() 检查 expiresAt，过期则清除

5.3.3 Session 生命周期

函数	行为
saveSession(user)	创建 Session，写入 localStorage
getSession()	读取并检查有效期，过期返回 null
clearSession()	从 localStorage 删除
isSessionValid()	getSession() !== null

5.3.4 初始化流程

auth.init() 在应用启动时调用：1. 从 localStorage 同步恢复 user（消除登录页闪烁）2. 后台异步从 Supabase 拉取最新用户信息 3. 调用 updateLastLogin() 更新最后登录时间 4. 若云端用户不存在 → 清除 session

5.4 密码修改

来源：src/stores/auth.ts:changePassword()



6. 数据同步机制

6.1 离线优先策略

来源：src/services/hybrid.ts、src/services/storage.ts

CleanNotes 采用 离线优先（Offline-First）架构：

写操作（如 upsertTask）

- ├── 1. 立即写入 localStorage（同步，保证本地数据即时一致）
- └── 2. 异步写入 Supabase
 - ├── 成功 → isOnline = true
 - └── 失败 → isOnline = false → pushDirtyOp() 加入脏队列

读操作（getTasks 为例）：- 本地有数据 → 直接返回本地（本地为 ground truth）- 本地为空（首次登录/新设备）→ 尝试从 Supabase 拉取

6.2 增量同步（30秒轮询）

来源：src/composables/useSync.ts

useSync() composable 启动

- ├── startHealthCheck() ← 每 15 秒检查 Supabase 连通性 + 重放脏队列
- └── startSyncLoop()
 - ├── 首次延迟 10 秒（避免与登录时 mergeFromCloud 冲突）
 - └── setInterval(runIncrementalSync, 30_000)

6.2.1 增量同步流程

runIncrementalSync() → fetchRemoteChanges()：

1. 并行拉取云端和本地数据（Tasks / DeletedTasks / AiMessages）
2. 计算差异：
3. **云端新增/更新**：云端有但本地无，或云端 updatedAt 更新 → updatedTasks
4. **云端已删除**：本地有但云端无 → deletedTaskIds
5. 排除脏队列中的待同步 ID（避免误判为远端删除）
6. 排除回收站中的 ID（防止已删除任务被恢复）
7. 排除墓碑中的 ID（防止永久删除任务被恢复）
8. 清理孤立记录（同时存在于 tasks 和 deleted_tasks 表的 ID）
9. 返回 RemoteChanges 对象，由 store 应用到 UI

6.2.2 健康检查

来源：src/services/hybrid.ts:189-205

每 15 秒：

- checkSupabaseHealth()
 - ├── GET /rest/v1/cleannote_tasks?limit=0（5秒超时）
 - ├── status < 500 → 可达
 - └── 其他 → 不可达

若可达且有脏队列 → syncDirtyOps()

6.3 全量合并（登录时）

来源：src/services/hybrid.ts:mergeFromCloud()

登录后调用，执行云端↔本地双向合并：

mergeFromCloud()
|

```

├── 并行拉取云端全量数据（5个数据集）
│   ├── getTasks() / getDeletedTasks() / getAiMessages()
│   └── getTimerConfig() / getAiConfig()
├── 并行读取本地最新数据（避免覆盖并行操作）
├── 合并 Tasks（排除回收站 + 墓碑）
│   ├── mergeRecords(local, cloud, compareUpdatedAt)
│   │   ├── 仅云端有 → 拉到本地
│   │   ├── 仅本地有 → 上传到云端
│   │   └── 都有 → updatedAt 新者胜出（平局合并字段）
│   └── 清理孤立记录（删除 tasks 表中已被 deleted_tasks 占用的 ID）
├── 合并 DeletedTasks（按 deletedAt 比较）
├── 合并 AiMessages（按 timestamp 比较）
├── TimerConfig / AiConfig（云端优先）
│   ├── 仅云端有 → 拉到本地
│   ├── 仅本地有 → 上传到云端
│   └── 都有 → 云端覆盖本地
├── saveLastSyncAt(now)
└── appendSyncLog({ type: 'full_merge' })

```

6.4 脏标记与重放

来源：src/services/hybrid.ts:139-271

6.4.1 脏队列结构

```

interface DirtyOp {
  type: 'upsert_task' | 'delete_task'
    | 'upsert_deleted_task' | 'delete_deleted_task'
    | 'upsert_ai_message' | 'delete_ai_message' | 'delete_all_ai_messages'
    | 'save_timer_config' | 'save_ai_config'
    | 'save_tasks' | 'save_deleted_tasks' | 'save_ai_messages'
  data?: any // 操作数据（upsert 时）
  id?: string // 操作 ID（delete 时）
}

```

存储 key: cleannotes_dirty_queue_{userId} (localStorage)

6.4.2 重放机制

syncDirtyOps() 在健康检查发现 Supabase 可达时触发：

1. 读取脏队列
2. 逐条重放操作
3. 成功的标记为已同步
4. 遇到首条失败 → 停止（可能是连通性问题）
5. 移除已成功的操作

6.5 墓碑机制

来源：src/services/hybrid.ts:54-137

6.5.1 目的

当任务被永久删除（从回收站清空或过期自动清理），其 ID 被记录为墓碑。防止以下场景：- cleannotes_tasks 表的 DELETE 请求失败 - 回收站记录也被删除 - 下次同步时云端无此 ID → 被误判为"远端已删除" → 但实际是本地操作未同步

6.5.2 实现

属性	值
存储 key	cleannotes_tombstones_{userId} (localStorage)
数据结构	{ [taskId]: UTC_ISO_timestamp }
最大保留期	90 天 (TOMBSTONE_MAX_AGE_DAYS = 90)
清理时机	用户切换时 (setUserId())

函数	行为
getTombstones()	返回所有墓碑 ID 的 Set
addTombstones(ids)	添加墓碑
removeTombstones(ids)	移除墓碑 (如恢复任务时)
cleanOldTombstones()	清理超过 90 天的墓碑

6.6 同步日志

来源: src/services/syncLog.ts

属性	值
存储 key	cleannotes_sync_logs_{userId} (localStorage)
最大条数	3 条 (FIFO 淘汰)
用途	SyncLogModal 弹窗展示

日志类型: full_merge | incremental | dirty_replay | error

此外, hybrid.ts 中还有一个内存同步日志 (syncLogs ref), 最多 20 条, 用于 UI 实时展示。

6.7 独立模块同步策略

除 hybridAdapter 管理的 Tasks/DeletedTasks/AiMessages/TimerConfig/AiConfig 外, 以下模块有独立的同步逻辑:

模块	文件	策略
Memo	memoStorage.ts	写: 本地+后台异步云端; 登录时 syncMemosFromCloud()
Todo	todoStorage.ts	同上模式, syncTodosFromCloud()
Growth	growthStorage.ts	写: 本地+防抖2秒后云端; syncGrowthFromCloud()
WeeklyReport	weeklyReportStorage.ts	同上模式, syncWeeklyReportsFromCloud()

6.7.1 Growth 特殊合并逻辑

来源: src/services/growthStorage.ts:140-212

Growth 合并采用"权威源"策略: - totalXp 高的一方为权威源 - 累积型字段 (streakDays、maxStreakDays、witheredDays) 取双方最大值 - XpEvent[] 按 id 合并去重, 时间戳新者胜出 - AchievementRecord[] 按 id 合并去重, unlockedAt 新者胜出

6.7.2 WeeklyReport AI 字段保护

来源: src/services/weeklyReportStorage.ts:88-93

合并周报时, 如果云端 aiSummary 缺失但本地有, 保留本地 AI 总结字段, 避免覆盖。

7. 数据迁移历史

7.1 迁移文件清单

序号	文件	内容	适用场景
1	supabase-schema.sql	完整数据库 Schema (v3)	新建数据库
2	migration-memos.sql	备忘录表 + icon/sort_order 列	v1.2.0 新增备忘录模块
3	migration-todos-growth.sql	待办表 + 养成系统表	v1.2.0 新增待办+养成
4	migration-weekly-reports.sql	周报表	v1.2.0 新增周报
5	migration-ai-summary.sql	周报增加 ai_summary 列	v1.3.0 AI 总结功能
6	migration-attachments-storage.sql	Storage Bucket + RLS	v1.3.0 附件存储 (未启用)
7	migration-password.sql	用户表增加密码字段	v1.3.0 密码登录
8	fix-supabase-timezone.sql	修复历史时区数据	一次性数据修复

7.2 迁移执行顺序

```
v3 基础 Schema (supabase-schema.sql)
├── v4 → v5 迁移
│   ├── ALTER TABLE cleannote_tasks ADD COLUMN in_progress_at
│   └── ALTER TABLE cleannote_deleted_tasks ADD COLUMN in_progress_at
├── migration-memos.sql (备忘录表)
│   └── ALTER TABLE ADD COLUMN icon, sort_order (幂等)
├── migration-todos-growth.sql (待办 + 养成)
├── migration-weekly-reports.sql (周报)
│   └── migration-ai-summary.sql (周报 AI 字段, 幂等)
├── migration-password.sql (密码字段, 幂等)
├── migration-attachments-storage.sql (Storage Bucket)
└── fix-supabase-timezone.sql (一次性修复, ⚠️ 需备份后执行)
```

7.3 v5 时间格式迁移

7.3.1 问题描述

旧版代码使用 toLocalISO() 生成无 Z 后缀的本地时间字符串 (如 CST 2026-07-01T12:34:00.000), PostgreSQL TIMESTAMPTZ 将无时区后缀的值当作 UTC 存储。导致数据库中的时间比真实 UTC 多了 8 小时 (+08 时区)。

7.3.2 修复方案

fix-supabase-timezone.sql 对受影响的时间字段减去 8 小时, 将"伪 UTC"修正为真实 UTC:

表	修复字段
cleannote_tasks	created_at、updated_at、completed_at、in_progress_at
cleannote_deleted_tasks	同上 + deleted_at
cleannote_todos	created_at、updated_at
cleannote_memos	created_at、updated_at
cleannote_ai_messages	timestamp
cleannote_users	last_login_at

7.3.3 前端防护

`normalizeTimestamp()` 函数在写入 Supabase 前对时间戳进行规范化：- 有 `z` 或时区偏移 → 原样返回 - 无时区后缀 (legacy) → 解析为本地时间，转 UTC ISO

7.4 幂等性说明

所有迁移脚本均设计为幂等 (IF NOT EXISTS、IF NOT EXISTS)，可安全多次执行。`fix-supabase-timezone.sql` 是唯一非幂等脚本，仅执行一次。

8. API 接口清单

8.1 基础信息

属性	值
Base URL	<code>https://ghkyhbx1tdxhkhpqltdr.supabase.co</code>
REST API 前缀	<code>/rest/v1/</code>
Storage API 前缀	<code>/storage/v1/</code>
认证方式	apikey header + <code>Authorization: Bearer</code> header
用户隔离	<code>x-user-id</code> header

8.2 请求头

apikey: {SUPABASE_KEY}
Authorization: Bearer {SUPABASE_KEY}
Content-Type: application/json
x-user-id: {currentUserId}
Prefer: return=minimal,resolution=merge-duplicates (upsert 时)

8.3 通用查询参数

PostgREST 查询语法 (通过 URL query string 传递):

参数	示例	说明
<code>{col}=eq.{val}</code>	<code>?user_id=eq.abc-123</code>	等值过滤
<code>{col}=eq.{val}&limit=1</code>	<code>?phone=eq.13800138000&limit=1</code>	限制返回条数
<code>order={col}. {asc\ desc}</code>	<code>?order=created_at.asc</code>	排序

8.4 通用 Prefer header

Prefer 值	说明
<code>return=representation</code>	返回插入/更新的完整行 (用于注册用户)
<code>return=minimal</code>	不返回 body (204, 用于大部分写操作)
<code>resolution=merge-duplicates</code>	upsert 语义: 主键冲突时合并而非报错

8.5 各表 API 端点

8.5.1 cleannote_users (用户)

操作	方法	URL	查询参数	Prefer
查找用户（手机号）	GET	/rest/v1/cleannote_users	?phone=eq. {phone}&limit=1	—
查找用户（含密码凭据）	GET	/rest/v1/cleannote_users	?phone=eq. {phone}&limit=1	—
注册用户	POST	/rest/v1/cleannote_users	—	return=representation
更新最后登录	PATCH	/rest/v1/cleannote_users	?id=eq. {userId}	—
更新昵称	PATCH	/rest/v1/cleannote_users	?id=eq. {userId}	—
设置密码	PATCH	/rest/v1/cleannote_users	?id=eq. {userId}	—

8.5.2 cleannote_tasks（任务）

操作	方法	URL	查询参数	Prefer
获取所有任务	GET	/rest/v1/cleannote_tasks	?user_id=eq. {uid}&order=created_at.asc	—
全量保存	POST	/rest/v1/cleannote_tasks	—	return=minimal,resolution=merge-duplicates
单条 upsert	POST	/rest/v1/cleannote_tasks	—	return=minimal,resolution=merge-duplicates
单条删除	DELETE	/rest/v1/cleannote_tasks	?id=eq. {id}	return=minimal
清空用户任务	DELETE	/rest/v1/cleannote_tasks	?user_id=eq. {uid}	return=minimal

8.5.3 cleannote_deleted_tasks（回收站）

操作	方法	URL	查询参数	Prefer
获取回收站	GET	/rest/v1/cleannote_deleted_tasks	?user_id=eq. {uid}&order=deleted_at.desc	—
全量保存	POST	/rest/v1/cleannote_deleted_tasks	—	return=minimal,resolution=merge-duplicates
单条 upsert	POST	/rest/v1/cleannote_deleted_tasks	—	return=minimal,resolution=merge-duplicates
单条删除	DELETE	/rest/v1/cleannote_deleted_tasks	?id=eq. {id}	return=minimal

8.5.4 cleannote_todos（待办）

操作	方法	URL	查询参数	Prefer
获取所有待办	GET	/rest/v1/cleannote_todos	?user_id=eq. {uid}&order=created_at.asc	—
单条 upsert	POST	/rest/v1/cleannote_todos	—	return=minimal,resolution=merge-duplicates
单条删除	DELETE	/rest/v1/cleannote_todos	?id=eq. {id}	return=minimal

8.5.5 cleannote_memos（备忘录）

操作	方法	URL	查询参数	Prefer
获取所有备忘	GET	/rest/v1/cleannote_memos	?user_id=eq. {uid}&order=created_at.desc	—
单条 upsert	POST	/rest/v1/cleannote_memos	—	return=minimal,resolution=merge-duplicates
单条删除	DELETE	/rest/v1/cleannote_memos	?id=eq. {id}	return=minimal

8.5.6 cleannote_growth（养成系统）

操作	方法	URL	查询参数	Prefer
获取养成数据	GET	/rest/v1/cleannote_growth	?user_id=eq. {uid}	—
upsert 养成数据	POST	/rest/v1/cleannote_growth	—	return=minimal,resolution=merge-duplicates

养成数据为单行 JSONB 聚合，无单条删除操作。

8.5.7 cleannote_weekly_reports（周报）

操作	方法	URL	查询参数	Prefer
获取所有周报	GET	/rest/v1/cleannote_weekly_reports	?user_id=eq. {uid}&order=week_start.desc	—
单条 upsert	POST	/rest/v1/cleannote_weekly_reports	—	return=minimal,resolution=merge-duplicates
单条删除	DELETE	/rest/v1/cleannote_weekly_reports	?id=eq. {id}	return=minimal

8.5.8 cleannote_timer_config（倒计时配置）

操作	方法	URL	查询参数	Prefer
获取配置	GET	/rest/v1/cleannote_timer_config	?user_id=eq. {uid}	—
保存配置	POST	/rest/v1/cleannote_timer_config	—	return=minimal,resolution=merge-duplicates

8.5.9 cleannote_ai_messages（AI 对话）

操作	方法	URL	查询参数	Prefer
获取消息	GET	/rest/v1/cleannote_ai_messages	?user_id=eq. {uid}&order=timestamp.asc	—
全量保存	POST	/rest/v1/cleannote_ai_messages	—	return=minimal,resolution=merge-duplicates
单条 upsert	POST	/rest/v1/cleannote_ai_messages	—	return=minimal,resolution=merge-duplicates
单条删除	DELETE	/rest/v1/cleannote_ai_messages	?id=eq. {id}	return=minimal
清空所有	DELETE	/rest/v1/cleannote_ai_messages	?user_id=eq. {uid}	return=minimal

8.5.10 cleannote_ai_config（AI 配置）

操作	方法	URL	查询参数	Prefer
获取配置	GET	/rest/v1/cleannote_ai_config	?user_id=eq.{uid}	—
保存配置	POST	/rest/v1/cleannote_ai_config	—	return=minimal,resolution=merge-duplicates

8.6 Storage API

操作	方法	URL	说明
上传附件	POST	/storage/v1/object/cleannote_attachments/{path}	FormData，路径：memo/{uid}/{ts}-{name}
获取公开URL	GET	/storage/v1/object/public/cleannote_attachments/{path}	直接访问，无需认证
删除附件	DELETE	/storage/v1/object/cleannote_attachments/{path}	404 静默忽略

8.7 错误处理

来源：src/services/supabase.ts:159-183 (request() 函数)

所有 API 请求通过统一的 request() 函数处理：

```
if (!res.ok) {
  const text = await res.text()
  throw new Error(`Supabase ${res.status}: ${text}`)
}
if (res.status === 204) return null
return res.json()
```

错误处理策略： - 写操作失败 → pushDirtyOp() 加入脏队列，等待重放 - 读操作失败 → 降级到 localStorage (isOnline = false) - 全量合并失败 → syncStatus = 'error'，记录同步日志 - 附件删除 404 → 静默忽略（文件可能已被删除）

8.8 健康检查端点

GET /rest/v1/cleannote_tasks?limit=0

使用 limit=0 的 GET 请求检查 Supabase 连通性（PostgREST 不支持对 /rest/v1/ 根路径发 HEAD）。5 秒超时，status < 500 视为可达。

附录 A：源文件索引

文件	职责
supabase-schema.sql	完整数据库 Schema（v3）
src/services/supabase.ts	Supabase REST API 封装、camelCase/snake_case 映射
src/services/auth.ts	认证服务（注册/登录/会话/密码）
src/services/hybrid.ts	混合存储适配器（离线优先+增量同步+脏队列+墓碑）
src/services/storage.ts	StorageAdapter 接口定义 + 适配器切换
src/services/local.ts	localStorage 适配器
src/services/memoStorage.ts	备忘录离线存储 + 云端同步
src/services/todoStorage.ts	待办离线存储 + 云端同步
src/services/growthStorage.ts	养成系统离线存储 + 防抖云端同步
src/services/weeklyReportStorage.ts	周报离线存储 + 云端同步

文件	职责
src/services/syncLog.ts	同步日志 (localStorage, 3条 FIFO)
src/composables/useSync.ts	增量同步 Composable (30秒轮询)
src/stores/auth.ts	认证状态管理 (Pinia store)
src/utils/time.ts	时间处理 (UTC ISO 规范 + normalizeTimestamp)
src/utils/crypto.ts	密码哈希 (PBKDF2-HMAC-SHA256)
src/types/index.ts	TypeScript 类型定义
migration-*.sql	各模块数据库迁移脚本
fix-supabase-timezone.sql	历史时区数据修复脚本